

Programação em Python

Unidade 1

Desenvolver Algoritmos Utilizando Lógica de Programação

Sessão 02

Pyhton

Desafio

- Qual a utilidade de Python hoje?
- Como é mantida a linguagem Python?
- Qual o site da linguagem?
- Onde podemos utilizar a linguagem?

Python

Características

- Linguagem simples e interpretada (script)
- Linguagem multiplataforma
- Possui muitos recursos
- Possui uma extensa biblioteca
- Diversos módulos são fornecidos por terceiros
- Existe diversas comunidades de programadores



Saiba mais

- Sobre a linguagem Python, visite:
- <https://www.python.org/> (site oficial)
- <https://wiki.python.org.br/PythonBrasil>
- <https://python.org.br/>

Python

Execução de programas

- Diversos editores permitem criar programas em Python
- Pode ser utilizado de forma interativa

Prompt de Comando - python

```
E:\Python>python
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Olá! Eu fui escrito em Python.")
Olá! Eu fui escrito em Python.
>>> _
```

Python

Operadores aritméticos

Operador	Descrição	Exemplo
+	Soma	$x = y + z$
-	Subtração	$x = y - z$
*	Multiplicação	$x = y * z$
/	Divisão	$x = y / z$
%	Módulo	$x = y \% z$
**	Exponenciação	$x = 5 ** 2$
//	Parte inteira da Divisão	$x = 3 // 2$

Python

Comentários e Tipos de dados simples

- Comentários:
 - Linha simples – # (símbolo “hash”)
 - Múltiplas linhas – """ (três aspas duplas)
- Tipos de dados simples
 - Numéricos
 - Inteiros / Inteiros longos
 - Ponto flutuante
 - Float
 - Complexos (parte real + parte imaginária)
 - Lógicos - True ou False
 - Caractere (char) - Letra ou símbolo utilizando aspas simples (' ') ou duplas (" ")
 - Strings – cadeias de caracteres utilizando aspas simples (' ') ou duplas (" ")

Python

Operadores de Strings

Operador / Caracteres especiais	Descrição
+	Concatenação
*	Repetição
\n	Nova linha – <i>New Line ou Enter (LF)</i> *
\r	Retornar cursor para o início da linha – <i>Carriage Return (CR)</i> *
\t	Tabulação – <i>Tab</i> *
\b	Backspace *
\\	Representa uma barra – \ *
\x41	Caractere onde código hexadecimal é igual a 41 – “A” <i>maiúsculo</i> *

Obs.: Os caracteres especiais não são imprimíveis.

Python

Tipos de dados compostos

- Listas
 - Coleções ordenadas de elementos, podendo ser modificadas, ou seja, são mutáveis
 - Utilizam colchetes como identificação – `[ ... , ... , ... ]`
- Tuplas
 - Semelhantes às listas, porém, são imutáveis, ou seja, não podem ser modificadas
 - Utilizam parênteses como identificação – `( ... , ... , ... )`
- Dicionários
 - Estruturas de dados que armazenam pares de chave-valor. A chave é única e permite a recuperação dos dados de forma eficiente
 - Utilizam chaves como identificação – `{ ... , ... , ... }`
- Conjuntos
 - São coleções de elementos não ordenadas e não duplicadas
 - Permitem realizar operações de união, interseção e diferença entre conjuntos
 - Utilizam chaves como identificação – `{ ... , ... , ... }`

Python

Tipos de dados compostos

- Exemplos de tipos de dados compostos:
 - Listas
 - [100, 300, 500]
 - ["fiat", "jeep", "toyota"]
 - Tuplas
 - (100, 300, 500)
 - ("fiat", "jeep", "toyota")
 - Dicionários
 - { "nome": "Fiat Prêmio", "modelo": "Luxo", "ano": 2011 }
 - Conjuntos
 - { 100, 300, 500 }
 - { "fiat", "jeep", "toyota" }

Python

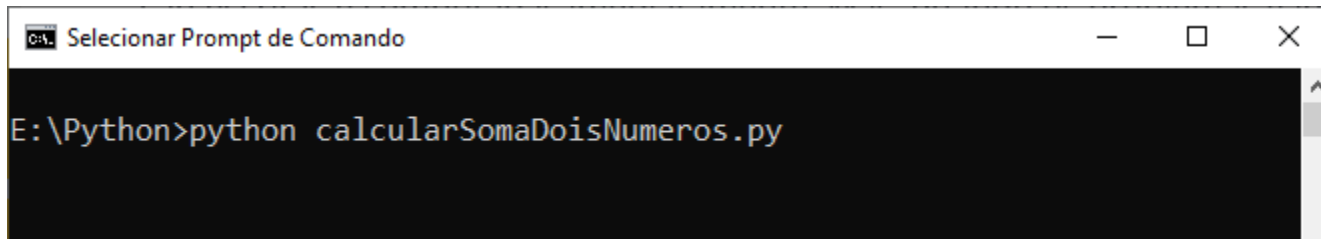
Variáveis

- Declaração de variáveis
 - `<nome-da-variável> = <valor-da-variável>`
- Nome
 - Letras
 - Símbolo “_”
 - “Case-sensitive”
 - Por convenção, utilizar letras minúsculas e separar as palavras com “_”
- Abrangência / Escopo
 - Somente pode ser usada no módulo do programa onde foi criada
- Atribuição múltipla
 - Ex.:
 - `a,b = 1,2`
 - `a,b,c,d = 'Maria', 100, 'Rua das Palmeiras, 20', 22887`

Python

Programas em Python

- Extensão do arquivo - “.py”
 - Exemplo
 - calcularSomaDoisNumeros.py
- Execução de um programa no shell

A screenshot of a Windows Command Prompt window. The title bar reads "Selecionar Prompt de Comando". The command prompt shows the directory "E:\Python" and the command "python calcularSomaDoisNumeros.py" being executed.

```
Selecionar Prompt de Comando
```

```
E:\Python>python calcularSomaDoisNumeros.py
```

Python

Entrada e Saída em Python

- Entrada
 - Comando/instrução – `input()`
 - Formato
`<nome-da-variável> = input( "Mensagem para o usuário" )`
 - Exemplo
`nome = input('Digite seu nome')`
`matricula = int ( input( 'Digite sua matrícula' ) )`
`salario = float ( input( 'Digite seu salário' ) )`

Obs.: int e float foram inseridos para realizar o “cast” dos valores, transformando a string lida do console em um primitivo inteiro (int) ou um ponto flutuante (float).

Python

Entrada e Saída em Python

- Saída
 - Comando ou instrução – `print()`
 - Formato

```
print ( valores, [ sep=' ', end=' ', file=sys.stdout ] )
```

 - `valores` – lista de mensagens e/ou variáveis separadas por vírgula
 - Itens opcionais
 - `sep` – indica o tipo de separador apresentado entre os valores
 - `end` – indica a string onde será concatenado o final do print, onde o padrão é “Enter” ou “\n”
 - `file` – saída para onde são enviados os valores, onde o padrão é “Console(sys.stdout)”

Python

Entrada e Saída em Python

- Saída

- Exemplos

```
print("Olá", "!", "Tudo", "bem?")
```

```
# Pular \n linha na mensagem
```

```
print("Primeira linha\nSegunda linha")
```

```
# print com separador _ entre os valores a serem exibidos
```

```
numero = 4
```

```
print("Quadrado de", numero, " = ", numero**2, sep='_')
```

```
# utilizando o end para não pular a linha após o print
```

```
print("imprimindo uma linha ", end=' ')
```

```
print("continuando a linha acima")
```


Algoritmos

Lembrando da construção de um algoritmo...

Passos a seguir:

1. Definir o problema.
2. Executar uma análise preliminar.
3. Criar uma solução.
4. Aplicar o teste de qualidade.
5. Refinar a solução (Alteração).
6. Apresentar o produto final.

Algoritmos

Lembrando das fases de execução...

1. Coleta de informações de que o algoritmo precisa.
2. Processamento.
3. Apresentação do resultado.

Algoritmos

Tipos de processamento

- Tipos de Processamento:
 - *Processamento Sequencial*
 - Passos executados uma após o outro sem desvios.
 - *Processamento Condicional*
 - Conjunto de instruções executado ou não.
 - A execução depende de uma condição.
 - A condição pode definir uma resposta verdadeira ou falsa.
 - *Processamento com Repetição*
 - Conjunto de instruções que é executado um determinado número de vezes.

Algoritmos

Processamento sequencial

- Passos executados uma após o outro sem desvios.

Ex.:

Apresentar o salário líquido de um funcionário que tem descontos iguais a 20%:

inicio

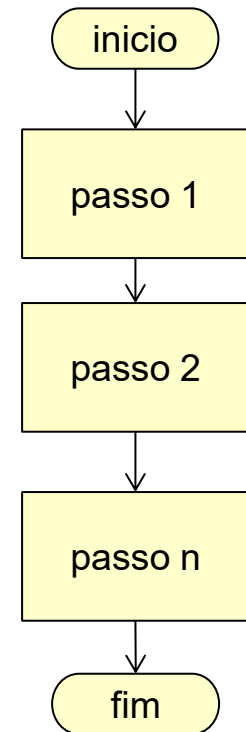
ler salarioBruto.

desconto <- salario * 0.2

salarioLiquido <- salarioBruto - desconto

escrever salarioLiquido

fim



Python

Algoritmo sequencial

Ex.:

Apresentar o salário líquido de um funcionário que tem descontos iguais a 20%:

```
salario_bruto = float(input("Informe o salário: "))
desconto = salario_bruto * 0.2
salario_liquido = salario_bruto - desconto

print(f"Salário líquido: {salario_liquido:.2f}")
```

Programador em Python

Botafogo



Python

Praticando

Problema:

- Crie um programa para cada exercício da lista de exercícios sequenciais.



Saiba mais

Referências sobre Python:

- <https://docs.python.org/pt-br/3/tutorial/index.html>

Algoritmos/Python

- Considerações finais.
- Agradecimentos.



Programador em Python

Botafogo



Até a próxima!

